Live Market Data Dashboard: Project Docs

Nov 6, 2026 · @William Welden

A Go service that holds one Alpaca real-time stock stream and fans it out to many browser clients over WebSockets, built to teach concurrency and to work as a resume project.

Overview

Your server keeps one upstream WebSocket open to Alpaca and serves any number of browser clients from it. Alpaca's limits force this design: many plans allow only one stream connection per user, and the free plan caps WebSocket subscriptions at 30 symbols. One shared connection is the only way to serve many viewers.

Goals

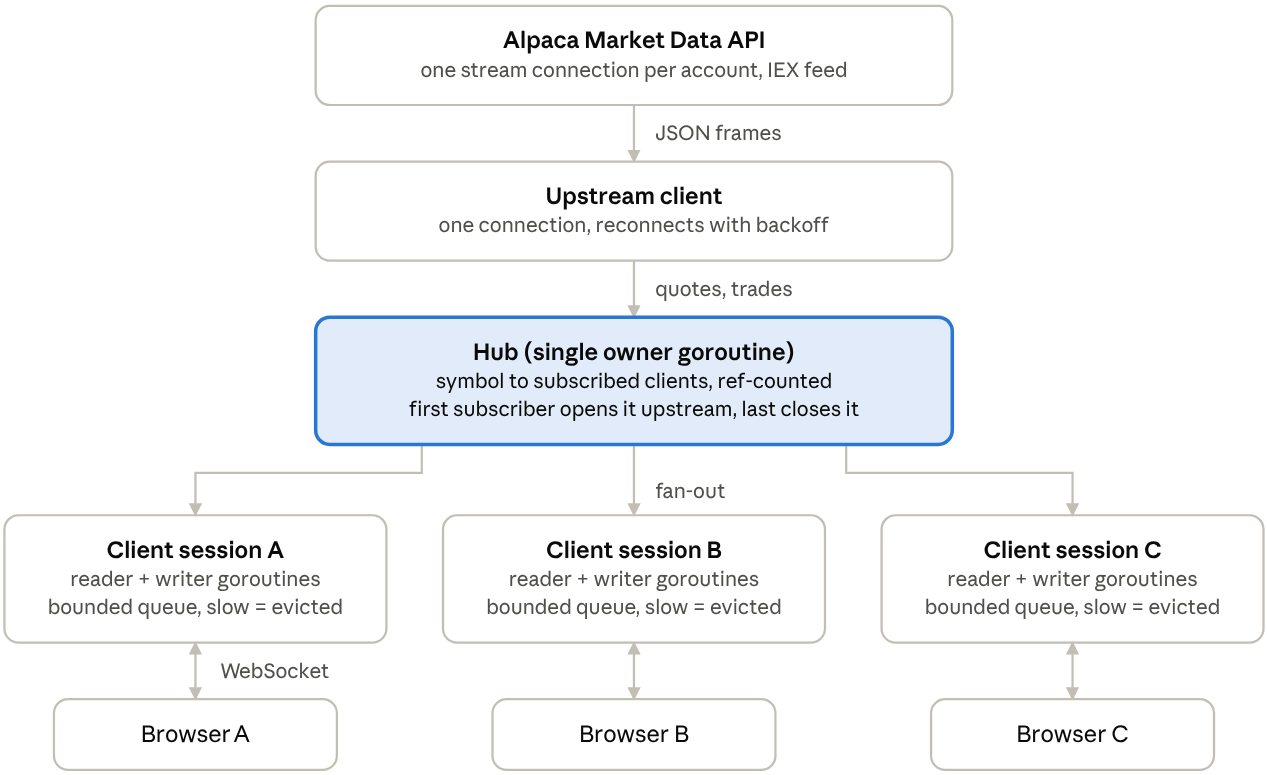
- Learn goroutines, channels, `sync`, `context` and WebSockets by building something real.
- Ship a deployed project with measured numbers (concurrent clients, latency) for a resume.
- Get comfortable with idiomatic Go: package layout, error handling, interfaces, tests.

Non-goals

- Placing trades. Alpaca's trading API is out of scope.
- A polished chart UI. A live table with a sparkline is enough.
- Long-term storage. A small in-memory buffer is optional; a database is not needed.

Architecture

One upstream connection feeds every browser through a single hub



architecture · Alpaca to hub to browsers

Only the upstream client talks to Alpaca; the hub owns all subscription state, and each browser gets its own queue so one slow client cannot stall the rest.

Alpaca API

The free Basic plan streams real-time stock data from IEX only, over one connection, for up to 30 symbols. That is enough for this project. Everything below comes from Alpaca's own docs, listed under Sources.

Setup

1. Create an Alpaca account. A paper-trading account works, and Basic is the default market data plan for both paper and live accounts.
2. Generate an API key and secret in the web dashboard, under API keys.

3. Export `APCA_API_KEY_ID` and `APCA_API_SECRET_KEY`. The Go SDK reads these names. Keep them out of the repo and commit a `.env.example` instead.

Plan limits (stocks)

Limit	Basic (free)	Algo Trader Plus ($99/month)
Real-time coverage	IEX only	All US exchanges
WebSocket symbols	30	Unlimited
Historical data	Since 2016, except the latest 15 minutes	Since 2016, no restriction
Historical API calls	200 per minute	10,000 per minute

IEX is one exchange, so its volume and top-of-book prices will not match the consolidated (SIP) tape. Label the feed "IEX" in your UI.

Stream endpoints

The URL pattern is `wss://stream.data.alpaca.markets/{version}/{feed}`.

Feed	Path	Use
Test	`v2/test`	Always on, even outside market hours. Subscribe to the symbol `FAKEPACA`. Use it for development.
IEX	`v2/iex`	Free real-time data. The feed to use on Basic.
Delayed SIP	`v2/delayed_sip`	Consolidated data, 15 minutes late. Check your plan before relying on it.
SIP	`v2/sip`	Real-time consolidated data. A feed outside your plan fails at authentication.

Regular US market hours are 9:30 a.m. to 4:00 p.m. Eastern, and the IEX feed is quiet outside them. Build against the test stream first.

Connection protocol

1. Open the WebSocket. The server replies `[{"T":"success","msg":"connected"}]`.
2. Authenticate within 10 seconds. Either send the headers `APCA-API-KEY-ID` and `APCA-API-SECRET-KEY` during the handshake, or send `{"action":"auth","key":"...","secret":"..."}` as a message. Success returns `[{"T":"success","msg":"authenticated"}]`.
3. Subscribe with `{"action":"subscribe","quotes":["AAPL","MSFT"],"trades":["AAPL"]}`. Send only the symbols you are adding; `unsubscribe` subtracts. Each reply lists your full current subscriptions.
4. Read messages. Every frame is a JSON array. Control messages hold one element, but data frames can hold many, so loop over the array.

The wildcard `*` subscribes to every symbol. Expect it to hit the 30-symbol cap on Basic, so list symbols explicitly. Alpaca buffers outgoing messages and may disconnect a slow client. Your server will face the same problem with slow browsers.

Message fields

Channel	`T`	Fields
Trades	`t`	`S` symbol, `i` trade ID, `x` exchange, `p` price, `s` size, `c` conditions, `t` timestamp (RFC 3339, nanoseconds), `z` tape
Quotes	`q`	`S` symbol, `bx` bid exchange, `bp` bid price, `bs` bid size, `ax` ask exchange, `ap` ask price, `as` ask size, `c` conditions, `t` timestamp, `z` tape
Minute bars	`b`	`S` symbol, `o` open, `h` high, `l` low, `c` close, `v` volume, `vw` VWAP, `n` trade count, `t` timestamp

Quote sizes are in round lots. Daily bars (`d`) and updated bars (`u`) share the bar layout, and minute bars arrive right after each minute mark. Subscribing to trades also subscribes you to trade corrections (`c`) and cancels or errors (`x`), so handle them or ignore them on purpose. A quote from the test stream looks like this:

```
{"T":"q","S":"FAKEPACA","bx":"O","bp":133.85,"bs":4,"ax":"R","ap":135.77,"as"
:5,"c":["R"],"z":"A","t":"2024-07-24T07:56:53.639713735Z"}
```

Errors

Code	Message	Your handling
400	invalid syntax	A bug in your message or a malformed symbol. Log it.
401	not authenticated	You subscribed before authentication finished. Fix the ordering.
402	auth failed	Wrong key or secret. Treat as fatal; do not retry in a loop.
403	already authenticated	You authenticated twice on one connection.
404	auth timeout	Authenticate sooner after connecting.
405	symbol limit exceeded	Your subscriptions are unchanged. Reject the browser client that asked.
406	connection limit exceeded	A second connection, or another app using your key. Keep exactly one upstream connection.
407	slow client	Your reader is too slow. Process faster or buffer more.
409	insufficient subscription	The feed is not in your plan. Treat as fatal and switch to IEX.
410	invalid subscribe action for this feed	You asked for a channel this stream does not offer.
500	internal error	Retry with backoff.

Optional: historical REST backfill

The historical API at `https://data.alpaca.markets/v2/...` takes the same two key headers. Basic allows 200 calls per minute and blocks the latest 15 minutes of consolidated data, so request the IEX feed. Use it to seed a sparkline when a browser first subscribes.

Terms of use

I did not find Alpaca's rules on redistributing market data in the pages I opened. Read their market data terms before putting a public demo online. A safe default is a read-only demo with a few fixed symbols.

Go SDK quickstart

Alpaca's official Go client (v3) handles authentication, subscriptions, compression and reconnects, so use it for the first version and spend your effort on the hub. Once it works, rewriting the upstream client by hand against the protocol above is a good exercise.

```
go mod init github.com/<you>/marketdash
go get github.com/alpacahq/alpaca-trade-api-go/v3@latest
```

How the stream client behaves

- `stream.NewStocksClient(feed, opts...)` builds the client. `Connect(ctx)` blocks until the first connection succeeds, then keeps reconnecting in the background. Call it once.
- `WithCredentials`, `WithQuotes`, `WithTrades` and `WithBars` set credentials and the first subscriptions with their handlers.
- `SubscribeToQuotes` and `UnsubscribeFromQuotes` change subscriptions after connecting. They block until they succeed or fail, and they cannot be called concurrently. One goroutine in your hub must own all upstream subscription changes.
- `WithReconnectSettings(limit, delay)` controls retries. A limit of 0 retries forever unless the error is irrecoverable, such as invalid credentials.
- `WithBufferSize` and `WithBufferFillCallback` handle a full receive buffer, which means your handlers are too slow. `WithProcessors(n)` with n above 1 breaks message ordering.
- `Terminated()` returns a channel that delivers the final error. A terminated client cannot be reused.

Minimal program

```
package main

import (
        "context"
        "log"
```

```
        "os"
        "os/signal"
        "time"

        "github.com/alpacahq/alpaca-trade-api-go/v3/marketdata"
        "github.com/alpacahq/alpaca-trade-api-go/v3/marketdata/stream"
)

func main() {
        ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt)
        defer stop()

        // The handler only hands quotes off; the hub reads from this
channel.
        quotes := make(chan stream.Quote, 1024)

        client := stream.NewStocksClient(
                marketdata.IEX, // confirm the constant name on pkg.go.dev
                stream.WithCredentials(os.Getenv("APCA_API_KEY_ID"),
os.Getenv("APCA_API_SECRET_KEY")),
                stream.WithQuotes(func(q stream.Quote) {
                        select {
                        case quotes <- q:
                        default: // buffer full: drop instead of blocking the
SDK
                        }
                }, "AAPL", "MSFT"),
                stream.WithReconnectSettings(0, time.Second),
        )

        if err := client.Connect(ctx); err != nil {
                log.Fatalf("connect: %v", err)
        }

        go func() {
                for q := range quotes {
                        log.Printf("%s bid %.2f ask %.2f", q.Symbol,
q.BidPrice, q.AskPrice)
                }
        }()

        select {
        case err := <-client.Terminated():
```

```
                log.Fatalf("stream terminated: %v", err)
        case <-ctx.Done():
                log.Println("shutting down")
        }
}
```

To develop outside market hours, point the client at the test stream with `WithBaseURL` and subscribe to `FAKEPACA`. Check the package docs for how the base URL combines with the feed path.

Verify before coding. I read most of the v3.11.0 stream package page on pkg.go.dev, which flagged itself as not the latest version. The cut-off part included the `Trade` struct, so check its field names and the feed constants there. Quote fields (`Symbol`, `BidPrice`, `AskPrice`) were confirmed.

Server design

The browser protocol is yours to define. Keep it small and independent of Alpaca's field names, so the vendor can change without breaking clients.

Browser protocol

JSON messages over a WebSocket at `/ws`. Each subscribed symbol delivers both its quotes and its trades. Prices below are illustrative.

Message	Direction	Example
subscribe	client to server	`{"action":"subscribe","symbols":["AAPL","MSFT"]}`
unsubscribe	client to server	`{"action":"unsubscribe","symbols":["MSFT"]}`
quote	server to client	`{"type":"quote","symbol":"AAPL","bid":187.10,"bidSize":2,"ask":187.12,"askSize":3,"ts":"2026-11-06T18:30:00.123Z"}`
trade	server to client	`{"type":"trade","symbol":"AAPL","price":187.11,"size":100,"ts":"2026-11-06T18:30:00.125Z"}`
status	server to client	`{"type":"status","upstream":"reconnecting"}`

Message	Direction	Example
error	server to client	`{"type":"error","code":"symbol_limit","message":"Too many symbols"}`

Alpaca's docs say 30 symbols on Basic but do not say whether a symbol subscribed to both quotes and trades counts once or twice. Test this early with the test stream. It decides how your hub counts the cap.

Concurrency design

- **One hub goroutine owns all subscription state.** Other goroutines send it messages over channels: register, unregister, subscribe, unsubscribe and market data. No mutex is needed, and the data race on the subscriber map cannot happen.
- **Each browser client has two goroutines.** A read loop parses actions and forwards them to the hub. A write loop drains the client's queue and is the only writer to the socket. `gorilla/websocket`, for one, allows a single concurrent writer.
- **Each client queue is a bounded channel**, for example 256 messages. The hub sends with `select` and `default`. A full queue means the client is slow, so the hub disconnects it. Conflation is the stretch goal.
- **One goroutine owns upstream subscription changes**, because the SDK forbids concurrent changes. It applies the hub's refcount decisions in order.
- **Everything stops through one `context`.** The server cancels it on Ctrl-C and each loop exits when it is done.

Repository layout

```
marketdash/
  cmd/
    server/main.go        wiring, config, signal handling, shutdown
    loadtest/main.go      opens N clients, reports throughput and latency
  internal/
    upstream/             Alpaca client wrapper; emits your own Quote and
Trade types
    hub/                  subscriptions, refcounts, fan-out (the core)
    client/               per-connection read and write loops
    protocol/             browser message types and JSON encoding
    fakefeed/             replays recorded messages for tests and demos
  web/                    static page: a live table of quotes
  Dockerfile
  .env.example
  README.md
```

The hub should depend on a small `Upstream` interface, not on the SDK. Tests then use `fakefeed`, and swapping the vendor later touches only `internal/upstream`.

Requirements

The must-haves are what make this a credible resume project instead of a tutorial clone. Each one has a testable "done when" so you can tick it off honestly.

Must have

Requirement	Done when
One upstream Alpaca connection shared by all clients	Ten browser tabs open, and Alpaca sees a single connection (no 406 errors)
Per-client subscribe and unsubscribe by symbol	A client receives only the symbols it asked for
Reference-counted upstream subscriptions	Alpaca subscribes when the first client wants a symbol and unsubscribes when the last one leaves
Symbol cap respected	The 31st distinct symbol is rejected with a clear error to the client, not an upstream 405

Requirement	Done when
Bounded send buffer per client	A deliberately stalled client is dropped or disconnected and never slows the others
Upstream reconnect with exponential backoff	After killing the network, the stream resumes, subscriptions are restored and clients see a status message
Graceful shutdown with `context`	Ctrl-C closes every connection and exits with no goroutine left running
Ping/pong heartbeats and read/write deadlines	A silently dead client is detected and cleaned up within a set time
Secrets from the environment	No key or secret anywhere in git history
`go test -race ./...` passes	Tests include the hub, with a fake upstream, run under the race detector

Should have

- [ ] A fake upstream server that replays recorded messages, so tests and demos run offline and outside market hours.
- [ ] Table-driven unit tests for message parsing and subscription refcounting.
- [ ] A load-test program that opens N clients and reports messages per second, p50 and p99 latency, and memory.
- [ ] Structured logging with `log/slog` and a `/healthz` endpoint.
- [ ] A Dockerfile and a GitHub Actions workflow running `go vet`, `go test -race` and a linter.
- [ ] A deployed demo with a live link (Railway or Fly.io), read-only and limited to a few symbols.
- [ ] A README with the architecture diagram, a benchmark table, and a "design decisions" section.

Stretch

- [ ] Prometheus metrics: connected clients, upstream reconnects, dropped messages, fan-out latency.

- [ ] Conflation: when a client is slow, keep only the latest quote per symbol instead of dropping at random.
- [ ] A hand-written upstream client replacing the SDK, using `context` and your own backoff.
- [ ] MessagePack on the browser side, with a measured bandwidth comparison against JSON.
- [ ] Historical backfill from the REST API to seed each client's sparkline.
- [ ] Horizontal scaling: two server instances sharing one upstream through Redis pub/sub.

Turning it into resume bullets

Only write numbers you measured. A template to fill in after the load test:

- Built a Go service that fans one Alpaca real-time stream out to **[N]** concurrent WebSocket clients, with p99 delivery latency under **[X] ms**.
- Designed bounded per-client buffers and slow-consumer eviction so one stalled client cannot degrade others; verified with the race detector and a load test.
- Implemented reference-counted upstream subscriptions, exponential-backoff reconnects, and graceful shutdown using `context`.

In interviews, expect questions on why one upstream connection, what happens when a client is slow, how you avoid races in the hub, and how shutdown works. The requirements above are chosen so you can answer each from your own code.

Learning path

Build in eight milestones, each a working program. For every concept, read it, break it on purpose, fix it, then write a test that would have caught the break. Do the exercises in small throwaway programs before you apply them to the project.

Concepts in plain words

Term	Plain meaning	Where it shows up here
Concurrency	Structuring a program as tasks that make progress independently. It does not require multiple cores.	The hub juggles many clients at once

Term	Plain meaning	Where it shows up here
Parallelism	Tasks literally running at the same instant on different cores.	Mostly incidental here; the work is I/O-bound
Async	Starting slow work (network, disk) and doing other things while it waits. Go has no `async` / `await`: you write blocking-looking code in goroutines and the runtime parks them while they wait.	Every socket read and write
Goroutine	A lightweight function running concurrently, started with `go f()`.	Two per browser client, one for the hub, one for upstream
Channel	A typed pipe between goroutines. Sending and receiving synchronize them; a buffered channel holds up to N items.	The hub's inbox and each client's send queue
`select`	Waits on several channel operations and runs whichever is ready. With a `default` case it never blocks.	The hub loop, and the "drop if the buffer is full" send
Race condition	Two goroutines touch the same memory, at least one writes, and nothing orders them. Results become unpredictable.	The map of symbol to subscribers
Mutex vs channels	A mutex guards shared state. The Go habit is to give state one owner goroutine and send it messages instead.	Choosing how the hub protects its subscription map
`context`	Carries cancellation and deadlines down through calls and goroutines.	Shutdown, and the lifetime of each connection
Backpressure	What happens when a producer is faster than a consumer. You must choose: block, buffer, drop or disconnect.	A slow browser while quotes keep arriving

Term	Plain meaning	Where it shows up here
Fan-out	Delivering one input to many consumers.	The hub sending each quote to every subscriber
WebSocket	A long-lived two-way connection that begins as an HTTP request and is upgraded. Data moves as frames, with ping/pong for liveness and a close handshake.	Both the upstream and the browser side
Exponential backoff	Waiting longer after each failure (1s, 2s, 4s), with a cap and random jitter, so retries do not stampede.	Upstream reconnect
Goroutine leak	A goroutine that never exits, usually blocked on a channel nobody will use again.	A client disconnecting without both of its goroutines stopping

Milestones

#	Build	Learn	Done when
0	A small Go module with tests	Packages, errors as values, interfaces, `testing`, `gofmt`, `go vet`	`go test` is green on a message-parsing function
1	A worker pool and a pipeline (throwaway)	Goroutines, channels, `select`, `sync.WaitGroup`	1,000 jobs processed by 5 workers with no lost results
2	A shared counter, broken then fixed (throwaway)	Data races, `sync.Mutex`, `sync/atomic`, `context` cancellation	`go test -race` fails on the broken version and passes on both fixes
3	A WebSocket echo server, then a chat room	HTTP upgrade, frames, ping/pong, close, one reader and one writer goroutine per connection	Three browser tabs chat in one room

#	Build	Learn	Done when
4	The hub with subscriptions and a fake upstream	Single-owner goroutine, reference counts, bounded queues, a drop policy	100 test clients get only their symbols at 1,000 messages per second, and a stalled client is evicted
5	Alpaca ingest	The SDK client, reconnects, the test stream, mapping vendor types to your own	Live `FAKEPACA` quotes reach a browser
6	Hardening	Heartbeats, deadlines, graceful shutdown, `log/slog`	Ctrl-C exits cleanly with no leaked goroutines
7	Measure and ship	Load testing, benchmarks, `pprof`, Docker, CI, deployment	The README benchmark table is filled in and the demo is live

Reading list

I listed these from memory and did not open them this session, so check that the links still work.

- A Tour of Go: the goroutines, channels and `select` sections.
- Effective Go: the concurrency section, including "share memory by communicating".
- Go Concurrency Patterns: Pipelines and cancellation: the pattern behind milestones 1 and 2.
- Data Race Detector: how to read `-race` output.
- RFC 6455, The WebSocket Protocol: skim the framing and closing sections.
- Rob Pike's talk "Concurrency Is Not Parallelism": the clearest short explanation of the first two terms above.
- A WebSocket library: pick one of `github.com/coder/websocket` or `github.com/gorilla/websocket` and read its examples.

Sources

Alpaca pages opened and checked on 2026-11-06. Plan limits and prices change, so recheck them before relying on them.

- About Market Data API: plan limits, authentication, historical API.
- WebSocket Stream: connection, authentication, subscriptions, errors.
- Real-time Stock Data: feeds, channels, message fields.
- Go stream package, v3.11.0: client options and methods. Read in part; see the verify note in the SDK section.

The concurrency design, requirements, milestones and reading list are my own recommendations, not from these pages.